```
# 1. Uber fare price prediction

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression from
sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score


# 1. READ DATASET
df = pd.read_csv("uber.csv")
print(df.head())
print("\nShape:", df.shape)
print("\nMissing Values:\n", df.isnull().sum())

# 2. PREPROCESSING
df.drop(["Unnamed: 0", "key"], axis=1, inplace=True, errors="ignore")

df["pickup_datetime"] = pd.to_datetime(
    df["pickup_datetime"], errors="coerce", utc=True
)
df["hour"] = df["pickup_datetime"].dt.hour df["day"]
= df["pickup_datetime"].dt.day df["month"] =
df["pickup_datetime"].dt.month df["year"] =
df["pickup_datetime"].dt.year df["weekday"] =
df["pickup_datetime"].dt.dayofweek

df.drop("pickup_datetime", axis=1, inplace=True) df.dropna(inplace=True)

df = df[df["fare_amount"] > 0]
df = df[df["passenger_count"] > 0] print("\nShape

after preprocessing:", df.shape)

# 3. OUTLIER DETECTION
Q1 = df["fare_amount"].quantile(0.25) Q3
= df["fare_amount"].quantile(0.75)

IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
```

```
print("\nLower Bound:", lower)
print("Upper Bound:", upper)

#  Boxplot
plt.figure(figsize=(8,4))
sns.boxplot(x=df["fare_amount"])
plt.title("Fare Amount Outliers")
plt.show()

# Remove outliers
df = df[
    (df["fare_amount"] >= lower) & (df["fare_amount"]
    <= upper)
]

# 4. CORRELATION
plt.figure(figsize=(10, 7))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()

# 5. FEATURES AND TARGET
X = df.drop("fare_amount", axis=1) y
= df["fare_amount"]


# 6. TRAIN TEST SPLIT
X_train, X_test, y_train, y_test = train_test_split( X,
    y,
    test_size=0.20,
    random_state=42
)

# 7. LINEAR REGRESSION
lr = LinearRegression()
lr.fit(X_train, y_train)

lr_pred = lr.predict(X_test)

lr_mae = mean_absolute_error(y_test, lr_pred)
lr_rmse = np.sqrt(mean_squared_error(y_test, lr_pred)) lr_r2
= r2_score(y_test, lr_pred)


# 8. RANDOM FOREST
rf = RandomForestRegressor(
    n_estimators=100,
    random_state=42,
```

```
    n_jobs=-1
)

rf.fit(X_train, y_train) rf_pred
= rf.predict(X_test)

rf_mae = mean_absolute_error(y_test, rf_pred)
rf_rmse = np.sqrt(mean_squared_error(y_test, rf_pred)) rf_r2 =
r2_score(y_test, rf_pred)

# 9. COMPARISON
results = pd.DataFrame({
    "Model": [
        "Linear Regression",
        "Random Forest"
    ], "MAE":
    [
        lr_mae,
        rf_mae
    ], "RMSE":
    [
        lr_rmse,
        rf_rmse
    ],
    "R2 Score": [
        lr_r2, rf_r2
    ]
})

print("\nMODEL COMPARISON")
print(results)
```

```
# 2.Email Spam Classification (KNN AND SVM)

import pandas as pd import
numpy as np

from sklearn.model_selection import train_test_split from
sklearn.neighbors import KNeighborsClassifier from
sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    precision_score, recall_score,
    f1_score, confusion_matrix,
    classification_report
)

import matplotlib.pyplot as plt
import seaborn as sns

# 1. READ DATASET
df = pd.read_csv("emails.csv")
print(df.head()) print("\nShape:",
df.shape) print("\nMissing
Values:")
print(df.isnull().sum().sum())


# 2. REMOVE UNNECESSARY COLUMN
df.drop(df.columns[0], axis=1, inplace=True)
print("\nShape after removing ID:", df.shape)


# 3. SEPARATE FEATURES AND TARGET
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

print("\nFeature Shape:", X.shape)
print("Target Shape:", y.shape)

print("\nClass Distribution:")
print(y.value_counts())

# 4. TRAIN TEST SPLIT

X_train, X_test, y_train, y_test = train_test_split( X,
    y, test_size=0.20,
    random_state=42,
    stratify=y
)


# 5. KNN CLASSIFIER
knn = KNeighborsClassifier(n_neighbors=5)
```

```
knn.fit(X_train, y_train) knn_pred
= knn.predict(X_test)


# KNN METRICS
knn_accuracy = accuracy_score(y_test, knn_pred)
knn_precision = precision_score(y_test, knn_pred)
knn_recall = recall_score(y_test, knn_pred) knn_f1 =
f1_score(y_test, knn_pred)

print("\n===== KNN RESULTS =====")
print("Accuracy :", knn_accuracy)
print("Precision:", knn_precision)
print("Recall    :", knn_recall)
print("F1 Score :", knn_f1)

print("\nClassification Report:")
print(classification_report(y_test, knn_pred))


# 6. KNN CONFUSION MATRIX
cm_knn = confusion_matrix(y_test, knn_pred)

plt.figure(figsize=(5, 4))
sns.heatmap(
    cm_knn,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.title("KNN Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# 7. SVM CLASSIFIER
svm = SVC(kernel="linear")
svm.fit(X_train, y_train) svm_pred
= svm.predict(X_test)

# SVM METRICS
svm_accuracy = accuracy_score(y_test, svm_pred)
svm_precision = precision_score(y_test, svm_pred)
svm_recall = recall_score(y_test, svm_pred) svm_f1 =
f1_score(y_test, svm_pred)

print("\n===== SVM RESULTS =====")
print("Accuracy :", svm_accuracy)
```

```
print("Precision:", svm_precision)
print("Recall    :", svm_recall)
print("F1 Score :", svm_f1)

print("\nClassification Report:")
print(classification_report(y_test, svm_pred))

# 8. SVM CONFUSION MATRIX
cm_svm = confusion_matrix(y_test, svm_pred)

plt.figure(figsize=(5, 4))
sns.heatmap(
    cm_svm,
    annot=True,
    fmt="d",
    cmap="Greens"
)

plt.title("SVM Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()


# 9. MODEL COMPARISON
results = pd.DataFrame({
    "Model": ["KNN", "SVM"],
    "Accuracy": [knn_accuracy, svm_accuracy],
    "Precision": [knn_precision, svm_precision],
    "Recall": [knn_recall, svm_recall],
    "F1 Score": [knn_f1, svm_f1]
})

print("\n===== MODEL COMPARISON =====")
print(results)
```

```
# PRACTICAL 3: BANK CUSTOMER CHURN PREDICTION

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

# 1. Read Dataset
df = pd.read_csv("Churn_Modelling.csv")

print(df.head())

# 2. Remove Unnecessary Columns
df.drop(
    ["RowNumber", "CustomerId", "Surname"],
    axis=1,
    inplace=True
)

# 3. Convert Categorical Data into Numerical
df = pd.get_dummies(
    df, columns=["Geography", "Gender"], drop_first=True
)

# 4. Separate Features and Target
X = df.drop("Exited", axis=1)
y = df["Exited"]

# 5. Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 6. Normalize Data
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 7. Build Basic Neural Network
model1 = MLPClassifier(
    hidden_layer_sizes=(10,),
    max_iter=300,
    random_state=42
)

model1.fit(X_train, y_train)

pred1 = model1.predict(X_test)
```

```
print("\nBasic Model Accuracy:")
print(accuracy_score(y_test, pred1))

# 8. Improved Neural Network
model2 = MLPClassifier(
    hidden_layer_sizes=(64, 32),
    activation="relu",
    solver="adam",
    learning_rate_init=0.001,
    max_iter=500,
    early_stopping=True,
    random_state=42
)

model2.fit(X_train, y_train)

pred2 = model2.predict(X_test)

# 9. Final Evaluation
print("\nImproved Model Accuracy:")
print(accuracy_score(y_test, pred2))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, pred2))

print("\nClassification Report:")
print(classification_report(y_test, pred2))
```

```
# PRACTICAL 4: GRADIENT DESCENT

import matplotlib.pyplot as plt

# Function
def function(x):
    return (x + 3) ** 2

# Derivative
def derivative(x):
    return 2 * (x + 3)

# Initial values
x = 2
learning_rate = 0.1
iterations = 50

x_values = []

# Gradient Descent Algorithm
for i in range(iterations):
    x_values.append(x)
    gradient = derivative(x)
    x = x - learning_rate * gradient
    print("Iteration:", i + 1, "x:", x, "y:", function(x))

# Final Result
print("\nLocal Minimum:")
print("x =", x)
print("y =", function(x))

# Plot Function
import numpy as np

X = np.linspace(-6, 4, 100)
Y = function(X)

plt.plot(X, Y, label="y = (x+3)^2")

# Plot Gradient Descent points
plt.scatter(x_values, [function(i) for i in x_values],
            color="red", label="Gradient Descent")

plt.xlabel("x")
plt.ylabel("y")
plt.title("Gradient Descent")
plt.legend()
plt.grid()
plt.show()
```

```
# 5.K-NEAREST NEIGHBORS ON DIABETES DATASET

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    confusion_matrix, accuracy_score,
    precision_score, recall_score
)

# 1. Load dataset
df = pd.read_csv("diabetes.csv")
print(df.head())

# 2. Separate features and target
X = df.drop("Outcome", axis=1)
y = df["Outcome"]

# 3. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 4. Scale the data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 5. Apply KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

# 6. Predict results
y_pred = knn.predict(X_test)

# 7. Calculate evaluation metrics
cm = confusion_matrix(y_test, y_pred)

accuracy = accuracy_score(y_test, y_pred)
error_rate = 1 - accuracy
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)

# 8. Display results
print("\nConfusion Matrix:")
print(cm)

print("Accuracy:", accuracy)
print("Error Rate:", error_rate)
```

```
print("Precision:", precision)
print("Recall:", recall)

# 9. Plot confusion matrix
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("KNN Confusion Matrix")
plt.show()
```

```
# PRACTICAL 6: K-MEANS CLUSTERING

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# 1. Read Dataset
df = pd.read_csv("sales_data_sample.csv", encoding="latin1")

print(df.head())
print(df.info())

# 2. Select Numerical Features
X = df[
    ["QUANTITYORDERED", "PRICEEACH", "SALES", "MSRP"]
]

# 3. Handle Missing Values
X = X.fillna(X.median())

# 4. Normalize Data
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# 5. Elbow Method
wcss = []

for i in range(1, 11):

    kmeans = KMeans(
        n_clusters=i,
        random_state=42,
        n_init=10
    )

    kmeans.fit(X_scaled)

    wcss.append(kmeans.inertia_)

# 6. Plot Elbow Graph
plt.plot(range(1, 11), wcss, marker="o")

plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")

plt.grid()
plt.show()

# 7. Apply K-Means
```

```
# Choose k based on the elbow in the graph
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

df["Cluster"] = kmeans.fit_predict(X_scaled)

# 8. Display Clusters
print("\nCluster Assignments:")
print(df[["SALES", "QUANTITYORDERED", "Cluster"]].head(10))

print("\nNumber of Customers/Orders in Each Cluster:")
print(df["Cluster"].value_counts())

# 9. Visualize Clusters
plt.scatter(
    X_scaled[:, 0],
    X_scaled[:, 2],
    c=df["Cluster"],
    cmap="viridis"
)

plt.xlabel("Quantity Ordered (Scaled)")
plt.ylabel("Sales (Scaled)")
plt.title("K-Means Clustering")
plt.show()
```